

Programación en C#

Unidad 4

Automatización,
optimización, supervisión
y solución basada en
requerimientos de
usuarios



UNIVERSIDAD
MAYOR
para espíritus emprendedores

TECMayor



Índice

Índice.....	1
Introducción	2
1. Creación de capas de desarrollo (MVC)	4
1.1 Capa vista o de presentación	6
1.2 Creación de objetos gráficos	6
1.3 Capa de control o controladora	11
1.4 Capa de negocio o modelo de datos	13
1.5 Capa de datos o conexión	13
2. Creación de Login	15
3. Control de excepciones	17
4. Activación y cierre de excepciones (throws, finally)	19
4.1 Instrucción try-catch-finally.....	20
Conclusión	22
Referencias bibliográficas	23



Introducción

El proceso de automatización de un modelo de negocio requiere el desarrollo de muchos módulos que tienen diversos fines en la Programación Orientada a Objeto (POO), unos relacionados con la presentación de los datos a los usuarios, otros con controlar los eventos del sistema y que están directamente relacionados con gestionar el modelo de datos y, finalmente, quien se conecta directamente con la base de datos.

La presentación de los datos y la información se desarrolla a través de las vistas de usuarios, que es soportada por los contenedores y componentes gráficos y textuales. El control de eventos y flujo de tráfico hacia la base de datos y la respuesta del sistema se desarrolla a través de componentes de tipo controladores, en cambio, el modelo lógico de datos mapea la información con las entidades físicas para su actualización y consulta.

Todo el proceso debe ser desarrollado de forma segura, para lo cual se requiere la implementación de un controlador de acceso denominado Login, el cual se encarga de generar la autenticación y el control de los usuarios a través de su perfil.

Dado que existen diversos módulos que utilizan diversos tipos de tecnologías, el sistema a desarrollar requiere un proceso que le permita seguir funcionando si algún componente tiene alguna interrupción en sus procesos, sin producir caídas anormales. Por consiguiente, requiere la implementación de controles de excepción, especialmente en las validaciones de datos y su conexión a la base de datos.

Esta implementación separada por módulos permite ordenar la construcción de la arquitectura a través de patrones creados para ejecutar tareas regulares del sistema. Algunos patrones se incluyen en las librerías de “Visual Studio” y sus versiones de “Frameworks”, y otros de tipo particularizado, implementados y adaptados por el equipo de desarrollo.

En el desarrollo del tipo cliente-servidor, cada una de las capas cubre una necesidad específica que parte desde la presentación, siguiendo al controlador y, finalmente, pasando la responsabilidad al esquema del mapeo de datos del servidor. Quien resuelve si la petición es válida y responde con un resultado válido, pero si existe algún error en la conexión, se activa una excepción que es reconocida por el sistema y reporta una señal de error.

En esta unidad hablaremos sobre las características más importantes del desarrollo de sistemas: automatización, optimización, supervisión y solución basada en los requerimientos de los usuarios. Con el propósito de contextualizarnos, examinemos el siguiente ejemplo:





La **implementación de un sistema de control de pólizas de una aseguradora**, por lo menos, debería incluir en su parte visual los datos de sus clientes y el catálogo de las pólizas que ofrece.

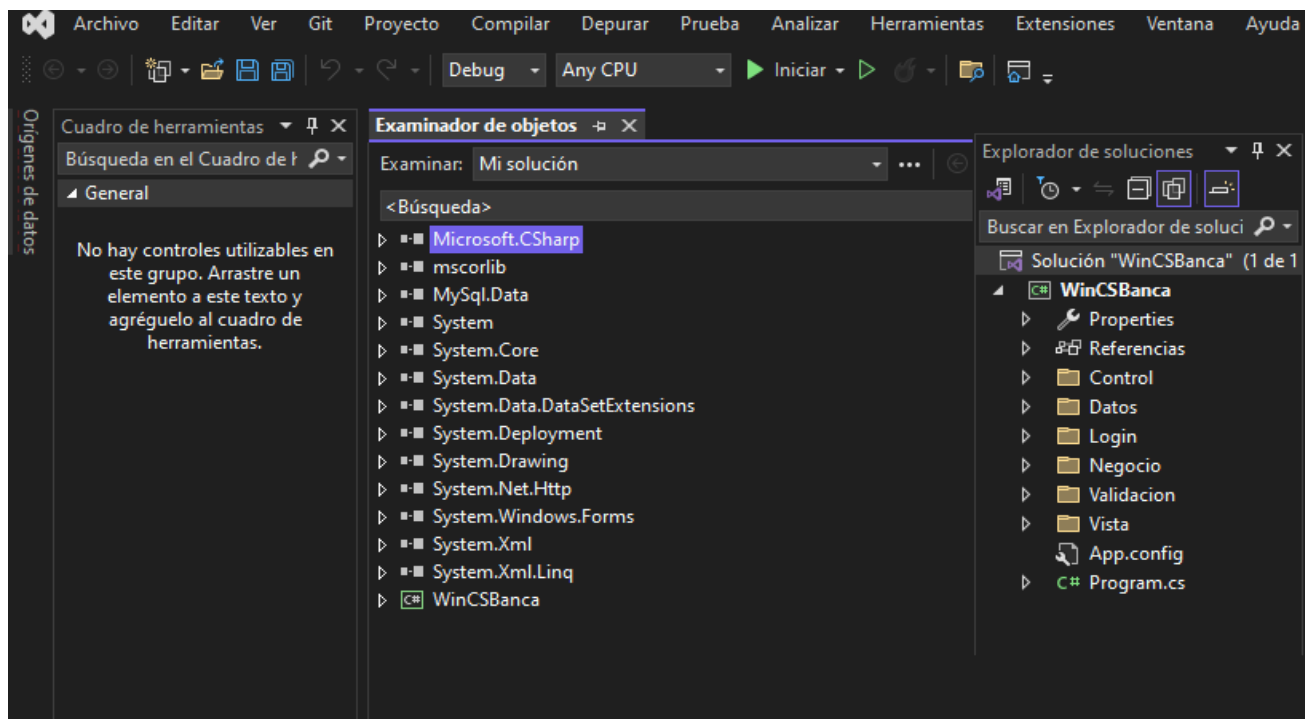
- En la parte **controladora**, debería haber una entidad que permita controlar las transacciones.
- De igual manera, en la parte del **modelo de datos**, sería necesario implementar las entidades “cliente” y “póliza”, con el fin de establecer el objeto de conexión que integra la parte cliente con el servidor de transacciones.
- Este **servidor** de transacciones debería contar con un repositorio que contenga al menos las entidades físicas, cliente y póliza, con el fin de almacenar los datos de las transacciones, con sus respectivos controles de excepciones.

1. Creación de capas de desarrollo (MVC)

Para la implementación, ordenamiento y modularización, es imperativo separar el sistema en al menos tres fases de desarrollo, a saber: **Modelo**, **Vista** y **Controlador** (MVC). La primera parte de ella debe incluir las entidades que brindan información a los usuarios. La segunda, el controlador, se encarga de dirigir los eventos y procesos de la aplicación y el modelo, evalúa el modelo de negocio en su parte lógica con la parte física que está vinculada directamente al tratamiento de los datos. La tercera parte sobre modularización representa la unidad de descomposición de software, por la que se seleccionan y agrupa instrucciones de programación que cumplen una función y cubren un servicio específico. Estos módulos se construyen utilizando operaciones o métodos, los cuales son bloques agrupados de código que realizan una función específica dentro del sistema.

En la figura 1 se muestra una pequeña aplicación separada por capas de desarrollo e implementación, en la cual cada carpeta dispone de módulos de clases específicos para una tarea identificada dentro del sistema, en la cual se pueden considerar las que integran el concepto MVC (Modelo Vista Controlador), aunque se pueden modularizar las que respaldan el sistema, en este caso: Login, Datos y Validación y que se explican en este apartado.

Figura 1. Implementación por capas de una aplicación, vista de la solución.



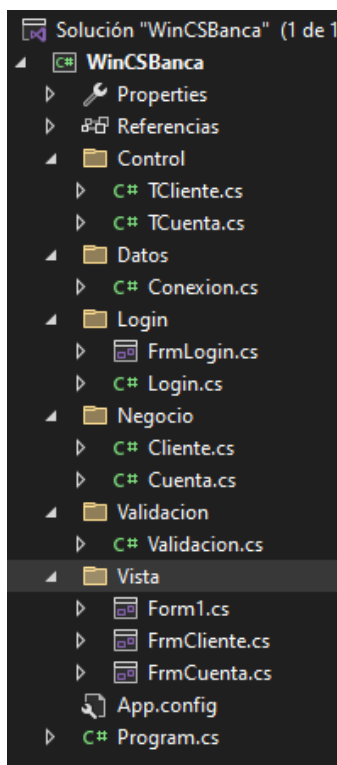
Fuente: Elaboración propia, 2024.



- **Capa o carpeta “Vista”:** posee las clases gráficas que implementan los formularios que contienen los contenedores y componentes para las interfaces de usuario.
- **Capa “Control” (Controlador):** integra las clases que implementan los métodos que permiten las transacciones con la base de datos.
- **Capa “Negocio” (Modelo):** integra las clases que se mapean con las tablas de base de datos y que contienen los constructores, accesadores y mutantes de cada una de ellas.
- **Capa “Login” (Enlace):** integra la interfaz de autenticación y el módulo de verificación que ve, valida o rechaza el ingreso a la aplicación.
- **Capa “Datos” (Conexión):** integra la clase que establece la comunicación con la base de datos a través de su método de conexión y controlador por excepciones.
- **Capa “Validación”** integra una clase que se encarga de crear métodos para validar diversos contextos cuando se capturan datos desde las vistas de los usuarios, tales como el RUT o una cadena de datos.

Como se muestra en la figura 2, una buena modularización de los requerimientos, incluye los aspectos realmente importantes requeridos por cada módulo de software. Por consiguiente, los modelos bien seleccionados y con una arquitectura modular adecuada reflejan los aspectos relevantes del sistema y permiten su implementación de manera eficiente.

Figura 2. Capas de una aplicación con sus dependencias.



Fuente: Elaboración propia, 2024.

1.1 Capa vista o de presentación

En el común de los casos es una clase límite, representada por una interfaz visual o hacia un sistema externo. En el caso de C# las capas de desarrollo se implementan a través de carpetas, en este caso “Vista”, donde se almacenan las clases contenedoras con sus componentes gráficos y textuales. En la actualidad, en el desarrollo fullstack se denomina front-end, lo que significa poner a disposición de los usuarios del sistema de interfaces gráficas de un alto grado de usabilidad, que sean de fácil entendimiento y operación y no produzca dualidades cognitivas.

Para ello se han desarrollado estándares que clarifican el modo y tipo de componentes a usar y los colores que no afecten a los usuarios. Visual Studio ha ido creando una variedad de estándares para apoyar la experiencia de usuario, aparte del concepto Windows Forms, el contexto [XAML](#) que permite crear instancias respaldadas por el estándar XML y el uso de sus componentes para el desarrollo de aplicaciones gráficas a través del framework o plataforma [WPF](#).



XAML (Extensible Application Markup Language)

También conocido como Lenguaje de marcado de aplicaciones extensible, es un lenguaje declarativo que puede inicializar objetos de sentido gráfico y establecer propiedades de objetos mediante una estructura de lenguaje que muestra relaciones jerárquicas entre varios objetos (Microsoft, 2023).

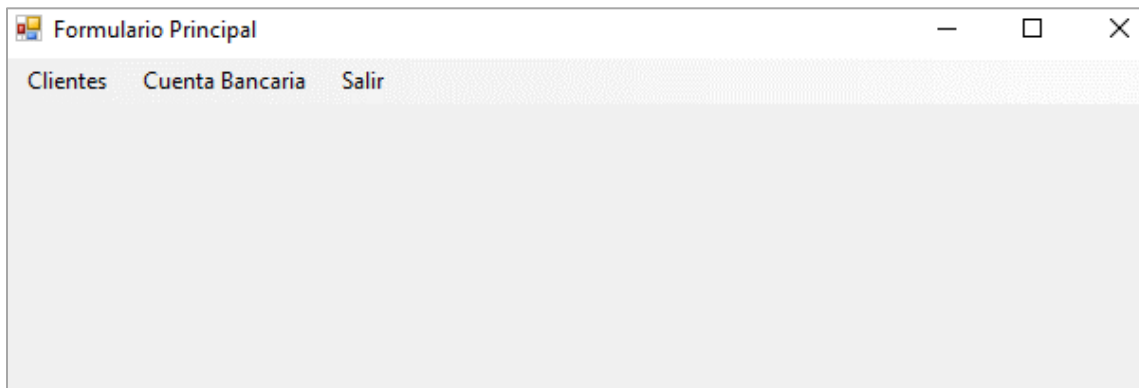
WPF (Windows Presentation Foundation)

Es un marco de interfaz de usuario que crea aplicaciones cliente de escritorio. La plataforma de desarrollo de WPF admite un amplio conjunto de características de desarrollo de aplicaciones, incluido un modelo de aplicación, recursos, controles, gráficos, diseños, enlace de datos, documentos y seguridad (Microsoft, 2023).

1.2 Creación de objetos gráficos

La creación de objetos gráficos que representen los requerimientos de los usuarios debe enfocarse en aspectos generales hacia los particulares. En la figura 3 se presentan los **elementos gráficos que se utilizan de arranques**, encapsulados en un menú de opciones que se dedica a levantar las unidades gráficas de usuario. En este caso, la emulación de un sistema de cuentas bancarias, en cuyo menú se puede apreciar: “Clientes”, “Cuenta Bancaria” y “Salir”.

Figura 3. Formulario principal de una aplicación.



Fuente: Elaboración propia, 2024.

A continuación, se puede apreciar su codificación:

```
namespace WinCSBanca
{
    public partial class FrmPpal : Form
    {
        public FrmPpal()
        {
            InitializeComponent();
        }
        private void salirDeLaAplicacionToolStripMenuItem_Click(object sender,
        EventArgs e)
        {
            MessageBox.Show("Saliendo de la aplicacion");
            Application.Exit();
        }
        private void gestionDeCuentasToolStripMenuItem_Click(object sender,
        EventArgs e)
        {
            FrmCuenta frmCuenta=new FrmCuenta();
            frmCuenta.Show();
        }
        private void gestionDeClientesToolStripMenuItem_Click(object sender,
        EventArgs e)
        {
            FrmCliente frmCliente = new FrmCliente();
            frmCliente.Show();
        }
    }
}
```

Para el desarrollo y descripción de los componentes, se enfocará en la descripción relacionada con el canal de los clientes. Si se activa el ítem del menú “Clientes”, se levanta el formulario de la figura 4 que exhibe las **funciones funcionales CRUD** (Create, Read, Update y Delete) para este módulo.



Figura 4. Formulario cliente con funcionalidades CRUD.

Formulario Cliente

Ingreso Cliente | Editar Cliente | Listar Clientes

Datos del Cliente

RUT

NOMBRE

APELLIDO

TELEFONO

Aceptar

Fuente: Elaboración propia, 2024.

La primera funcionalidad que se aprecia es el **ingreso de datos**, seguida por la **edición y despliegue de datos**. La funcionalidad de edición da soporte a la búsqueda de un registro, la modificación de algún atributo del registro o su eliminación desde la tabla cliente. En la figura 5 podemos observar una subventana de **búsqueda de datos**, que de acuerdo con el RUT genera una búsqueda en la tabla cliente que puede ser exitosa o errónea, en el supuesto de que se pueda ver esta búsqueda y la presentación de la búsqueda en la ventana de edición del registro cliente. Esta acción exitosa deja disponible el registro para su modificación o eliminación a través de los botones implementados para cada efecto.

Figura 5. Panel de edición de datos CRUD.

Formulario Cliente

Ingreso Cliente | Editar Cliente | Listar Clientes

Busqueda de Datos del Cliente

RUT

Buscar

Editor de Datos del Cliente

RUT

NOMBRE

APELLIDO

TELEFONO

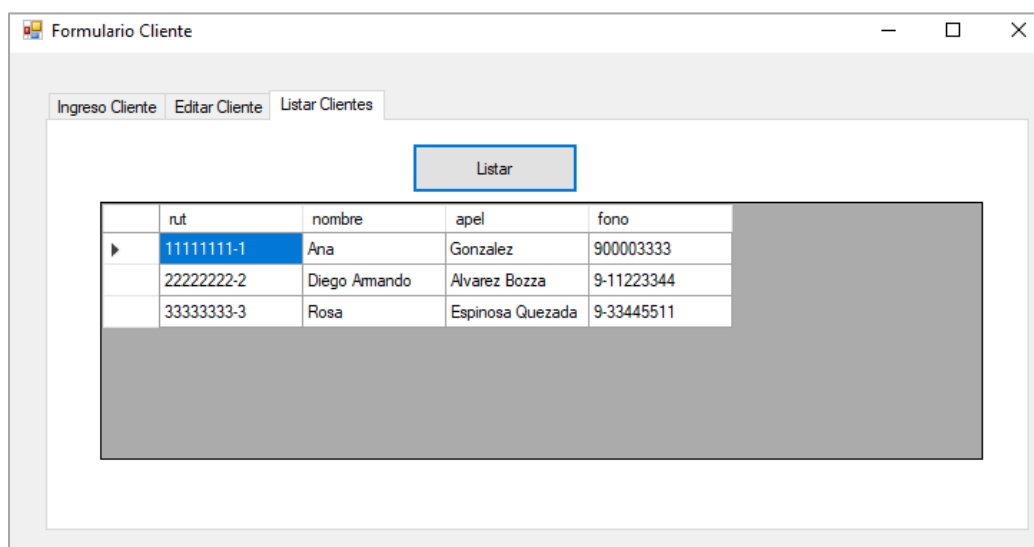
Modificar

Eliminar

Fuente: Elaboración propia, 2024.

Otra una funcionalidad importante que es la acción de **despliegue de todos los datos** que contiene una tabla para aspectos de generación de listado, efectuar filtrados por algunos de los atributos de los clientes, en este caso, el despliegue total del contenido del cliente, tal como se ilustra en la figura 6.

Figura 6. Panel de despliegue de datos.



	rut	nombre	apel	fono
▶	11111111-1	Ana	Gonzalez	900003333
	22222222-2	Diego Amando	Alvarez Bozza	9-11223344
	33333333-3	Rosa	Espinosa Quezada	9-33445511

Fuente: Elaboración propia, 2024.

A continuación, se muestra el código que se encarga de generar las peticiones y respuestas del lado del cliente:

```
namespace WinCSBanca
{
    public partial class FrmCliente : Form
    {
        public FrmCliente()
        {
            InitializeComponent();
        }

        private void btnAceptar_Click(object sender, EventArgs e)
        {
            Validacion validacion = new Validacion();
            Boolean existe = validacion.validarRut(txtRut.Text);
            if (existe == true)
            {
                MessageBox.Show("RUT CORRECTO...!", "AVISO DE SISTEMA",
                MessageBoxButtons.OK, MessageBoxIcon.Information);
                Cliente cliente = new Cliente();
                TCliente tCliente = new TCliente();
                cliente.rut = txtRut.Text;
                cliente.nombre = txtNombre.Text;
                cliente.apel = txtApel.Text;
                cliente.fono = txtFono.Text;
            }
        }
    }
}
```

```

        int resp = tCliente.ingresarCliente(cliente);
        if (resp > 0)
        { MessageBox.Show("Cliente ingresado en forma correcta....!",
"AVISO DE SISTEMA", MessageBoxButtons.OK, MessageBoxIcon.Information); }
        else
        { MessageBox.Show("Cliente con problema de ingreso....!", "AVISO
DE SISTEMA", MessageBoxButtons.OK, MessageBoxIcon.Information); }
    }
    else
        MessageBox.Show("¡RUT INCORRECTO...VERIFIQUE!", "AVISO DE
SISTEMA", MessageBoxButtons.OK, MessageBoxIcon.Information);
    }

private void btnListar_Click(object sender, EventArgs e)
{
    grillaClientes.DataSource = TCliente.listarClientes();//mvc4
}

private void btnBuscar_Click(object sender, EventArgs e)
{
    TCliente tCliente = new TCliente();
    Cliente cliente = new Cliente();
    string rut = txtRutBuscar.Text;
    cliente=tCliente.buscarCliente(rut);
    txtRutEd.Text = cliente.rut;
    txtNombreEd.Text = cliente.nombre;
    txtApelEd.Text = cliente.apel;
    txtFonoEd.Text = cliente.fono;
}

private void btnEliminar_Click(object sender, EventArgs e)
{
    TCliente tCliente = new TCliente();
    string rut = txtRutBuscar.Text;
    int resp=tCliente.eliminarCliente(rut);
    if (resp > 0)
    { MessageBox.Show("Cliente Elimina en forma correcta....!", "AVISO DE
SISTEMA", MessageBoxButtons.OK, MessageBoxIcon.Information); }
    else
    { MessageBox.Show("Cliente No Existe...verifique!", "AVISO DE
SISTEMA", MessageBoxButtons.OK, MessageBoxIcon.Information); }

}

private void btnModificar_Click(object sender, EventArgs e)
{
    Cliente cliente = new Cliente();
    TCliente tCliente = new TCliente();
    cliente.rut = txtRutEd.Text;
    cliente.nombre = txtNombreEd.Text;
    cliente.apel = txtApelEd.Text;

```



```

        cliente.fono = txtFonoEd.Text;
        int resp=tCliente.modificarCliente(cliente);
        if (resp > 0)
        { MessageBox.Show("Cliente Modificado en forma correcta....!", "AVISO
DE SISTEMA", MessageBoxButtons.OK, MessageBoxIcon.Information); }
        else
        { MessageBox.Show("Cliente con problema de actualizacion....!", "AVISO
DE SISTEMA", MessageBoxButtons.OK, MessageBoxIcon.Information); }
    }

```

1.3 Capa de control o controladora

En esta capa se implementan las clases encargadas de generar las transacciones y la comunicación entre la presentación y los datos. Se considera como una capa intermedia que recibe y dirige las peticiones que salen de la capa de presentación o vista, y encapsula el modelo de datos de cada clase y lo envía hacia la base de datos como un mapeo entre entidades lógicas y físicas.

Para cada acción de tipo CRUD, se implementa una [sentencia de tipo DML](#) que da soporte a cada servicio sobre la tabla de base de datos, tales como: Insert, Update, Delete y k. Una vez que envía la sentencia SQL, se espera su resultado por el motor de base de datos y retorna una retroalimentación hacia la interfaz que gatilla el evento. A continuación, se presenta el código de los métodos que se implementa en cada módulo:

```

using MySql.Data.MySqlClient;

namespace WinCSBanca
{
    class TCliente
    { //Triggers
        public int ingresarCliente(Cliente cliente)
        {
            int resp=0;
            MySqlConnection conexion = Conexion.abrirConexion();//Singleton
            MySqlCommand orden = new MySqlCommand(string.Format("INSERT INTO
CLIENTE(RUT,NOMBRE,APEL,FONO)
VALUES('{0}','{1}','{2}','{3}')",cliente.rut,cliente.nombre,cliente.apel,cliente.
fono), conexion);
            resp = orden.ExecuteNonQuery();
            conexion.Close();
            return resp;
        }

        public int modificarCliente(Cliente cliente)
        {
            int resp = 0;

```



1.4 Capa de negocio o modelo de datos

En esta capa se implementan las clases que tienen como objetivo mapear entre las entidades lógicas de la aplicación y las entidades físicas del modelo de base de datos. Estas son las entidades que se encapsulan en las sentencias generadas en el controlador.

Estas clases se usan como moldes o modelos para hacer las muestras que se envían y rescatan de la base de datos. Se compone de atributos, métodos constructores, accedadores y mutantes. A continuación, se puede apreciar el código:

```
using System;
namespace WinCSBanca
{
    class Cliente
    {
        public string rut { get; set; }
        public string nombre { get; set; }
        public string apel { get; set; }
        public string fono { get; set; }

        public Cliente() { }

        public Cliente(string rut, string nombre, string apel, string fono) {
            this.rut = rut;
            this.nombre = nombre;
            this.apel = apel;
            this.fono = fono;
        }
    }
}
```

1.5 Capa de datos o conexión

En esta capa se implementa la clase que establece el enlace y la comunicación entre la aplicación y el motor de la base de datos. Para ello, codifica un método especializado que encapsula una clase de conexión que viene integrada al paquete de conexión de MySQL y que considera un conjunto de parámetros que permite determinar el servidor, la base de datos, usuario y clave que la gestiona. Del mismo modo, la conexión controlada por el uso de excepciones en caso de falla de la conexión. A continuación, se puede apreciar el código:



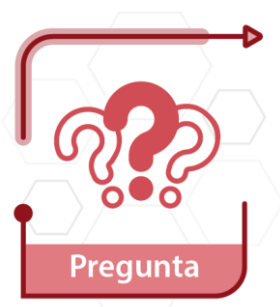
```

using System;
using MySql.Data.MySqlClient;
namespace WinCSBanca
{
    public class Conexion
    { //Singleton de Conexion
        public static MySqlConnection abrirConexion()
        {
            MySqlConnection canal = null;
            try
            {
                canal = new
                MySqlConnection("server=127.0.0.1;database=bdbanca;Uid=root;pwd=");
                canal.Open();
            }
            catch (Exception ex)
            {
                MessageBox.Show("CONEXION FALLADA...VERIFIQUE..." + ex.ToString());
            }
            return canal;
        }
    }
}

```

Para el aspecto de conexión a la base de datos, es preciso tener presente algunos aspectos metodológicos para brindar soporte a este aspecto de comunicación entre el cliente y el servidor:

- En primer lugar, se debe cargar el controlador de la base de datos “MySql.Data.MySqlClient”.
- Luego el componente controlador define el tipo de base de datos que se utilizará, en este caso MySQL, con el cual se debe crear un objeto de conexión ([MySqlConnection](#)). Esta acción o evento permitirá abrir la conexión con la base de datos.
- Una vez definida la base de datos y establecida la conexión esta información se pasa a los módulos de transacciones, los cuales crean sentencias de control que permiten efectuar acciones sobre la base de datos usando instrucciones SQL, que comúnmente permiten ingresar, eliminar, modificar y mostrar datos.



Pregunta

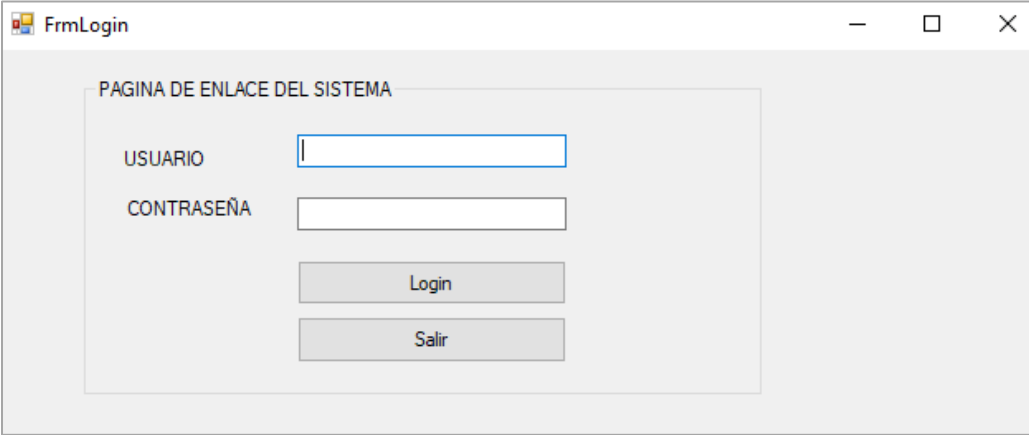
¿Cuál es la característica relevante del desarrollo por capas en aplicaciones de tipo cliente-servidor?

Se modulariza la arquitectura del sistema con el objetivo de separar la lógica de negocio de la lógica de diseño. Esto es, diseñar aplicaciones separando entre el modelo de datos, la lógica de aplicación, y la interfaz de usuario.

2. Creación de Login

La ventana de enlace al sistema oLogin” es la encargada de dar seguridad de acceso a los sistemas, con el propósito de evitar que usuarios no autorizados y no registrados en el sistema tengan acceso a sus funcionalidades. El módulo de Login se encarga de la autenticación del usuario, para lo cual comprueba en tiempo de ejecución que el nombre de usuario y contraseña sean correctos y válidos para su acceso al sistema, como se muestra en la figura 7.

Figura 7. Ventana de Login.

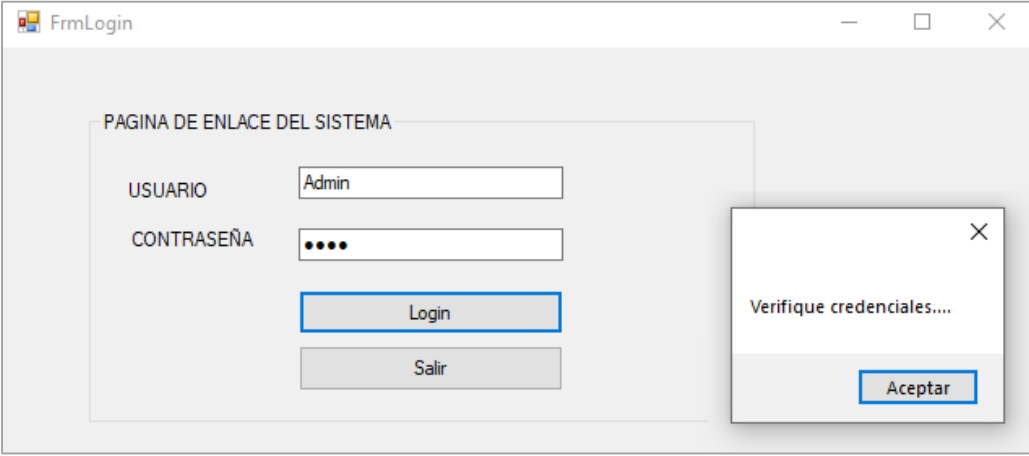


The screenshot shows a Windows-style window titled "FrmLogin". Inside, there is a container labeled "PAGINA DE ENLACE DEL SISTEMA". Within this container, there are two text input fields: the first is labeled "USUARIO" and the second is labeled "CONTRASEÑA". Below these fields are two buttons: "Login" and "Salir".

Fuente: Elaboración propia, 2024.

Si ambos atributos ingresados por el usuario (usuario y clave) en el *FrmLogin* y enviados a validar coinciden con el campo usuario y clave, se genera la instanciación de la interfaz principal y su posterior levantamiento. Sin embargo, si existe un error en las credenciales de acceso, la ventana generará un mensaje de error en el acceso, tal y como se muestra en la figura 8.

Figura 8. Error en credenciales de ingreso en la ventana de Login.



The screenshot shows the same "FrmLogin" window as in Figure 7. In this instance, the "USUARIO" field contains the text "Admin" and the "CONTRASEÑA" field contains four dots. The "Login" button is highlighted with a blue border. An error dialog box is overlaid on the right side of the window. The dialog box has a title bar with a close button (X) and contains the text "Verifique credenciales...." and an "Aceptar" button.

Fuente: Elaboración propia, 2024.



El código de implementación de FrmLogin es:

```
namespace WinCSBanca
{
    public partial class FrmLogin : Form
    {
        public FrmLogin()
        {
            InitializeComponent();
        }
        private void btnLogin_Click(object sender, EventArgs e)
        {
            Boolean existe = false;
            Login login = new WinCSBanca.Login();
            existe = login.validarUsuario(txtUsuario.Text,txtClave.Text);
            if (existe)
            {
                this.Hide();
                FrmPpal frmpp = new FrmPpal();
                frmpp.Show();
            }
            else MessageBox.Show("Verifique credenciales...");
        }

        private void btnSalir_Click(object sender, EventArgs e)
        {
            MessageBox.Show("Saliendo del LOGIN");
            Dispose();
        }
    }
}
```

El código de implementación de la página Login, que se puede apreciar en su codificación interna, indica que realiza una llamada de evento a una clase denominada Login. Dicha clase tiene codificado un método que se encarga de validar el usuario y su clave con respecto al registro almacenado en la tabla de control.

El código de la clase Login es:

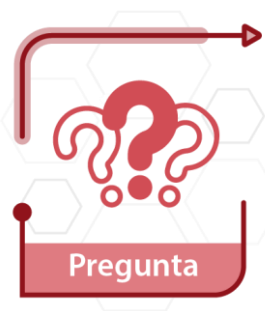
```
using System;
using MySql.Data.MySqlClient;
namespace WinCSBanca
{
    class Login
    {
        public Boolean validarUsuario(String usuar, String clave)
        {
            Boolean existe = false;
            MySqlConnection conexion = Conexion.abrirConexion();//Singleton
```



```

        MySqlCommand orden = new MySqlCommand("SELECT * FROM CONTROL WHERE
USUARIO='"+usuar+"' AND CLAVE='"+clave+"' ",conexion);
        MySqlDataReader lector = orden.ExecuteReader();
        if (lector.Read()) existe = true;
        else existe = false;
        return existe;
    }
}

```



Pregunta

¿Cuál es la inferencia de la implementación de Login de acceso en el desarrollo de aplicaciones?

Permitir el acceso al sistema solo al personal autorizado, de acuerdo con su perfil, para gestionar las funcionalidades que generan transacciones y manipulan datos. Este esquema permite operar con seguridad y mantener un alto nivel de integridad de los datos almacenados en el gestor de base de datos.

3. Control de excepciones

El manejo de excepciones permite a los desarrolladores implementar aplicaciones tolerantes a fallas y errores para la creación de sistemas robustos y que pueda seguir ejecutándose el programa sin verse afectado por el problema.

La administración de excepción es un concepto o filosofía de gestión en los sistemas, cuyo principio se fundamenta en la premisa de que solo es necesario intervenir o actuar en situaciones anormales o excepcionales. Este principio se aplica al desarrollo de aplicaciones con el fin de minimizar caídas en los sistemas en casos de corrupción de datos, errores en los mensajes de datos y pérdida de conexión entre cliente y servidor. A continuación, se presenta un reporte de excepción al no encontrar levantado el servidor de MySQL.

```

CONEXION FALLADA...VERIFIQUE...MySQL.Data.MySqlClient.MySQLException (0x80004005): Unable to connect to any of the specified MySQL hosts. ----> System.Net.Sockets.SocketException (0x80004005): No se puede establecer
una conexión ya que el equipo de destino denegó expresamente dicha conexión 127.0.0.1:3306
en System.Net.Sockets.Socket.InternalEndConnect(IAsyncResult asyncResult)
en System.Net.Sockets.Socket.EndConnect(IAsyncResult asyncResult)
en System.Net.Sockets.TcpClient.EndConnect(IAsyncResult asyncResult)
en System.Threading.Tasks.TaskFactory`1.FromAsyncCoreLogic(IAsyncResult iar, Func`2 endFunction, Action`1 endAction, Task`1 promise, Boolean requiresSynchronization)
en MySql.Data.MySqlClient.NativeDriver.<>c.<OpenAsync>b__38_0(Exception ex)
en System.AggregateException.Handle(Func`2 predicate)
en MySql.Data.MySqlClient.NativeDriver.<OpenAsync>d__38.MoveNext()
--- Fin del seguimiento de la pila de la ubicación anterior donde se produjo la excepción ---
en System.Runtime.CompilerServices.TaskAwaiter.ThrowForNonSuccess(Task task)
en System.Runtime.CompilerServices.TaskAwaiter.HandleNonSuccessAndDebuggerNotification(Task task)
en MySql.Data.MySqlClient.Driver.<OpenAsync>d__76.MoveNext()
--- Fin del seguimiento de la pila de la ubicación anterior donde se produjo la excepción ---
en System.Runtime.CompilerServices.TaskAwaiter.ThrowForNonSuccess(Task task)
en System.Runtime.CompilerServices.TaskAwaiter.HandleNonSuccessAndDebuggerNotification(Task task)
en MySql.Data.MySqlClient.Driver.<CreateAsync>d__74.MoveNext()
--- Fin del seguimiento de la pila de la ubicación anterior donde se produjo la excepción ---
en System.Runtime.CompilerServices.TaskAwaiter.ThrowForNonSuccess(Task task)
en System.Runtime.CompilerServices.TaskAwaiter.HandleNonSuccessAndDebuggerNotification(Task task)
en MySql.Data.MySqlClient.MySqlPool.<GetPooledConnectionAsync>d__30.MoveNext()
--- Fin del seguimiento de la pila de la ubicación anterior donde se produjo la excepción ---
en System.Runtime.CompilerServices.TaskAwaiter.ThrowForNonSuccess(Task task)
en System.Runtime.CompilerServices.TaskAwaiter.HandleNonSuccessAndDebuggerNotification(Task task)
en MySql.Data.MySqlClient.MySqlPool.<CreateNewPooledConnectionAsync>d__31.MoveNext()
--- Fin del seguimiento de la pila de la ubicación anterior donde se produjo la excepción ---
en System.Runtime.CompilerServices.TaskAwaiter.ThrowForNonSuccess(Task task)
en System.Runtime.CompilerServices.TaskAwaiter.HandleNonSuccessAndDebuggerNotification(Task task)
en MySql.Data.MySqlClient.MySqlPool.<TryToGetDriverAsync>d__34.MoveNext()
--- Fin del seguimiento de la pila de la ubicación anterior donde se produjo la excepción ---
en System.Runtime.CompilerServices.TaskAwaiter.ThrowForNonSuccess(Task task)
en System.Runtime.CompilerServices.TaskAwaiter.HandleNonSuccessAndDebuggerNotification(Task task)
en MySql.Data.MySqlClient.MySqlPool.<GetConnectionAsync>d__35.MoveNext()
--- Fin del seguimiento de la pila de la ubicación anterior donde se produjo la excepción ---
en System.Runtime.CompilerServices.TaskAwaiter.ThrowForNonSuccess(Task task)
en System.Runtime.CompilerServices.TaskAwaiter.HandleNonSuccessAndDebuggerNotification(Task task)
en System.Runtime.CompilerServices.TaskAwaiter.ValidateEnd(Task task)
en MySql.Data.MySqlClient.MySqlConnection.<OpenAsync>d__94.MoveNext()
--- Fin del seguimiento de la pila de la ubicación anterior donde se produjo la excepción ---
en System.Runtime.CompilerServices.TaskAwaiter.ThrowForNonSuccess(Task task)
en System.Runtime.CompilerServices.TaskAwaiter.HandleNonSuccessAndDebuggerNotification(Task task)
en MySql.Data.MySqlClient.MySqlConnection.Open()
en WinCSBanca.Conexion.abrirConexion() en C:\Users\admin\Desktop\ProyectosCS\WinCSBanca\Capas\WinCSBanca\WinCSBanca\Datos\Conexion.cs:línea 14

```

Aceptar

De esta manera, la administración por excepción evita que se pierda tiempo y recursos monitoreando de manera innecesaria cada uno de los procesos, permitiendo que el sistema se centre en los aspectos críticos que necesitan una toma de decisión por parte del módulo que es afectado por impresiones de datos precisos y rutinas poco relevantes.



Si se requiere validar algún proceso que es importante en un sistema y reducir los grados de integridad de datos, es posible aplicar dicho concepto. Uno de dichos procesos es la validación del RUT o la patente de un vehículo y que es un dato único y necesario para almacenamiento de datos de personas o de vehículos u otra entidad que la requiera por su criticidad.

A continuación, es posible apreciar la codificación y su método de validación de RUT, cuya rutina es verificada y controlada por la acción de la directiva try-catch:

```

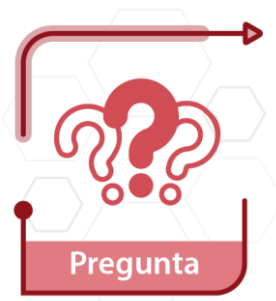
using System;
namespace WinCSBanca
{
    class Validacion
    {
        public bool validarRut(string rut)

```

```

{
    bool validac = false;
    try
    {
        rut = rut.ToUpper();
        rut = rut.Replace(".", "");
        rut = rut.Replace("-", "");
        int rutAux = int.Parse(rut.Substring(0, rut.Length - 1));
        char dv = char.Parse(rut.Substring(rut.Length - 1, 1));
        int m = 0, s = 1;
        for (; rutAux != 0; rutAux /= 10)
        {
            s = (s + rutAux % 10 * (9 - m++ % 6)) % 11;
        }
        if (dv == (char)(s != 0 ? s + 47 : 75))
        {
            validac = true;
        }
    }
    catch (Exception)
    {
    }
    return validac;
}
}

```



Pregunta

¿Cómo podríamos asegurar el control de excepciones en el desarrollo de aplicaciones?

Proporcionar a las aplicaciones rutinas que permitan brindar robustez a sus funcionalidades, con el objetivo de que sean tolerantes a fallos y mantenga sus procedimientos habituales sin verse afectado por caídas o inestabilidades en sus operaciones.

4. Activación y cierre de excepciones (throws, finally)

Como se mostró en el apartado 3 de este apartado, las instrucciones try, catch y throw se usan para trabajar con excepciones. Donde la instrucción throw permite producir y lanzar una excepción. En contraste, la instrucción try permite detectar y controlar (capturar) las excepciones que pueden surgir durante la ejecución de un bloque o rutina de código.

En el código siguiente código podremos ver cómo la instrucción `throw` produce una excepción, que puede ser utilizada cuando se requiere aceptar solo valores positivos mayores a 0 en una captura de datos o donde se enfoquen los aspectos de validación.

```
if (cantidad <= 0)
{
    throw new ArgumentOutOfRangeException(nameof(cantidad), "Cantidad debe ser mayor a 0 y de valor positivo.");
}
```

4.1 Instrucción try-catch-finally

Las sentencias `try`, `catch` y `finally` (detección, captura y cierre) pueden usarse en un proceso para la comprobación de errores, y permiten probar bloques de código y controlar lo que ocurre si se produce alguna excepción en tiempo de ejecución.

Utilizarlas evitará que la excepción termine el proceso principal que está ejecutando y muestre el mensaje de error estándar al usuario, pero esto significa que se tendrá que manejar lo que suceda después, generando un cierre del proceso o pasar a otra rutina evitando que el funcionamiento del sistema se ponga inestable o pueda efectuar acciones erráticas. En una instrucción `try-catch-finally`, el bloque `finally` se ejecuta cuando el control sale del bloque `try`.

El control puede dejar el bloque `try` como resultado de las siguientes situaciones:

- Ejecución normal.
- Ejecución de una instrucción de salto (es decir, `return`, `break`, `continue` o `goto`).
- Propagación de una excepción fuera del bloque `try`.

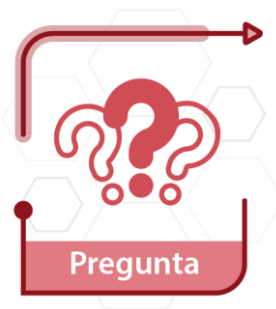
Se utiliza el bloque *finally* para restablecer el estado de un objeto antes de que el control deje el método, con el fin de controlar las excepciones que pueden producirse durante la ejecución del bloque `try` y especificar el código que se debe ejecutar cuando el control sale de la instrucción `try-catch`. Cuando un bloque `catch` controla una excepción, el bloque `finally` se ejecuta después de la ejecución del bloque `catch`.



Un **bloque *finally*** sirve para asegurarse de que se ejecute cierto código, independientemente de si ocurre una excepción controlada o no ocurre ninguna excepción. En general, estos bloques de código se usan para **liberar recursos que se asignaron en el bloque `try`** para cerrar archivos, tablas, procesos o si están trabajando con componentes de conexión para cerrar la comunicación entre aplicaciones.

A continuación, en el siguiente código podremos ver que el bloque finally cumple con la función de cerrar el archivo que se abre en el bloque try. Observe que se comprueba el estado del identificador de archivos antes de cerrar el archivo. Si el bloque try no puede abrir el archivo, el manipulador de archivos sigue teniendo el valor null, y el bloque finally no intenta cerrarlo. En lugar de eso, si el archivo se abre correctamente en el bloque try, el bloque finally cierra el archivo abierto (BillWagner, 2023).

```
FileStream? file = null;
FileInfo fileinfo = new System.IO.FileInfo("./file.txt");
try
{
    file = fileinfo.OpenWrite();
    file.WriteByte(0xF);
}
finally
{
    // Comprueba si hay valores nulos porque es posible que OpenWrite haya
    // fallado.
    file?.Close();
}
```



Pregunta

¿Cuál es el aporte en las funcionalidades del sistema que tienen las directivas de control de excepción en su aspecto de lanzamiento, captura y cierre de proceso?

Se utilizan bloques de código de control de excepción para liberar los recursos que se asignaron en el bloque para cerrar contenedores, conexiones u otro proceso antes de una caída anormal de un sistema, devolviendo los recursos al sistema operativo.



Conclusión

En esta unidad se abordó el proceso de automatización de un modelo de negocio que involucra toda una metodología de desarrollo que requiere la generación de diferentes módulos, desde que se inicia su implementación hasta establecer la comunicación con el gestor de base de datos, que es el repositorio que almacena el resultado de las transacciones y mantiene la persistencia de los datos que gestiona la aplicación.

Los elementos gráficos de interfaces de usuarios permiten diseñar diferentes estilos de ventanas, teniendo en cuenta los componentes que se integran a los contenedores que cubren los requerimientos de los usuarios para ingresar, modificar, eliminar y buscar datos en el sistema. Este conjunto de librerías utilizadas para diseñar las interfaces gráficas brinda soporte a mejoras en lo visual, portabilidad y comportamiento para aportar a la usabilidad de la aplicación que debe tenerse en cuenta al momento de la entrega del sistema a producción.

El desarrollo de las aplicaciones requiere un cierto nivel de seguridad, ya que ellas manipulan datos. Por lo tanto, la aplicación computacional debe iniciar sesión con los atributos de usuario y clave, que permitan validar y verificar sus credenciales. Si una de las opciones es errónea, debe emitir un mensaje de datos incorrecto; en caso contrario, indica ingreso correcto y permite operar las funcionalidades del sistema.

Construir las aplicaciones de tipo cliente-servidor involucra el desarrollo de la base de datos, programar las sentencias de conexión, las instrucciones de comunicación con el gestor de datos, que permitan obtener los resultados al insertar y mostrar datos a nivel de interfaz de usuarios, de acuerdo con lo solicitado por el requerimiento.

Lo anterior requiere adoptar una metodología que permita ordenar la codificación por diversas capas de desarrollo, teniendo en cuenta los aspectos de validación y control de excepciones para las tareas más exigentes del sistema a desarrollar. Esta metodología mejora la modularidad, la escalabilidad, el mantenimiento y la facilidad para llevar a cabo pruebas de aceptación del sistema, lo que propicia una mayor seguridad y flexibilidad en la implementación de nuevos servicios o funcionalidades.



Referencias bibliográficas

Anandmeg. (2023). *Aplicación Hola mundo con WPF en C#* - Visual Studio (Windows).

<https://learn.microsoft.com/es-es/visualstudio/get-started/csharp/tutorial-wpf?view=vs-2022>

BillWagner. (2023). *Control de excepciones* (Guía de programación de C#). Microsoft.com.

<https://learn.microsoft.com/es-es/dotnet/csharp/fundamentals/exceptions/exception-handling>

BillWagner. (2023). *Instrucciones de control de excepciones: throw y try, catch, finally* - C# reference.

Microsoft.com. <https://learn.microsoft.com/es-es/dotnet/csharp/language-reference/statements/exception-handling-statements>

Ceballos Sierra, F. J. (2007). *Microsoft C#. Curso de programación*. RA-MA Editorial. ISBN: 978-84-9964-396-0. México, Alfaomega.

Cerezo López, Y., Peñalba Rodríguez, O., & Caballero Roldán, R. (2007). *Iniciación a la programación en C#*. Las Rozas: Delta Publicaciones. España. Delta Publicaciones. ISBN: 9788496477537.

Ferguson, J., Patterson, B., Beres, J. Boutquin, P. y Gupta, M. (2003) *La Biblia de C#*. Madrid. España.

https://norbertomn.wordpress.com/wp-content/uploads/2014/09/anaya1-la-biblia-de-c_libro-programacion-espaaol_muestras_gratis_para_que_pruebes_en_tu_cas.pdf

Jwmsft. (2024). *Introducción a XAML - UWP applications*. <https://learn.microsoft.com/es-es/windows/uwp/xaml-platform/xaml-overview>



using System;
using System.Data;

namespace MyProject

{

public class Customer



UNIVERSIDAD
MAYOR

para espíritus emprendedores

TECMayor
